nekStab Documentation

Release 1.0.0

© 2020, Ricardo Frantz

April 06, 2021

List of Tables iii

1 Synopsis 1

2 Useful links 3
2.1 Installing and running . . . 3
2.2 Parameters . . . 4
2.3 Adapting your case . . . 5
2.4 Programming best practices . . . 6
2.5 Tutorials . . . 6
2.6 References . . . 8
2.7 Source Code . . . 8
2.8 Contributing . . . 9
2.9 Changelog on Nek5000 . . . 9

Index 11

Table 2.1: `STABILITY` keys in the `.par` file 4
Table 2.2: `BASE_FLOW` keys in the `.pars` file 5

Chapter 1

Synopsis

nekStab (Stability for Nek5000) is an open-source toolbox for massively-parallel eigenproblem and base flow computation developed for the Nek500 solver. It aims at solving any fluid flow of an incompressible, Newtonian fluid that can benefit from a Krylov-Schuer eigensolver. It supports natively 2D and 3D meshes with all sorts of boundary conditions and optional scalar transport equations. It aims to help users and developers …

Chapter 2

Useful links

- View nekStab on GitHub
- Suggestions for new features and bug report

2.1 Installing and running

2.1.1 Prerequisites

Ubuntu – Debian Distros

```
sudo apt-get -y install libmpich-dev libopenblas-dev cmake m4
```

Mac OS

```
install https://brew.sh/ and Xcode's Command Line Tools
brew install install gcc mpich openblas m4
brew install --cask cmake
(it also works on ARM-based procs via Rosetta 2)
```

(Optional packages)

```
#python
pip install tornado scipy

#other
code fortran-language-server paraview git dvipng htop

#code
krvajalm.linter-gfortran
nimda.deepdark-material
ms-vscode-remote.vscode-remote-extensionpack
hansec.fortran-ls
yukiuuh.vscode-modern-fortran-formatter
```

2.1.2 Cloning the repository

Clone the repository and Nek5000 as a submodule:

```
git clone --depth=50 --branch=master https://github.com/ricardofrantz/nekStab.git
cd nekStab
git submodule update --init --recursive
```

Run **(sudo) vim $HOME/.bashrc** and add the following:

```
export NEKSTAB_SOURCE_ROOT=$HOME/nekStab
export NEK_SOURCE_ROOT=$NEKSTAB_SOURCE_ROOT/Nek5000
export PATH=$NEK_SOURCE_ROOT/bin:$PATH
```

Compile Nek5000 tools:

```
cd ~/nekStab/Nek5000/tools
./maketools all
```

Outputs | ``building genmap … done

building gencon … done building genbox … done building n2to3 … done building reatore2 … done building nekmerge … done building prenek … done building postnek … done building nekamg_setup … done building gmsh2nek … done building exo2nek … done building cgns2nek … done``

Move to a specific example, for instance: **cd examples/1cyl**

```
./cmpile.sh all
nekbmpi 1cyl 4
```

2.2 Parameters

2.2.1 Current form of input via uparams

- uparam(1) = computation mode

  0: DNS, 1:stabilization, 3 : stability with direct, 3.2:stability with adjoint, 3.3:stability with transientGrowth
- uparam(2) = restart vector for stability (optional)
- uparam(3) = stabilzation technique (1:SFD)
- uparam(4) = target mode estimated Strouhal
- uparam(5) = target mode estimated growth rate
- uparam(6) =
- uparam(7) =
- uparam(8) = sponge side left length (e.g. 1)
- uparam(9) = sponge side right length (e.g. 5)
- uparam(10) = sponge strenght (>0 to activate e.g. 1.7)

2.2.2 Future update via parser

Stability File (.pars)

Table 2.1. `STABILITY` keys in the `.par` file

Key	Value(s)	Description

`eigenSolver`	`(none)`
	`arnoldi`
	`Krylov-Schur`
	`custom`
		Eigenvalue solver

evolOperator	(direct) adjoint transientGrowth optimalFrequency	Evolution operator
baseFlow	(steady) mean periodic	Base flow type
``forcing``	(none) prescribed reynoldsStess loadFromFile	Optional forcing for evolOperator
baseFlow	usric loadFromFile periodic	Base flow source
inSeed	noise symmetric loadFromFile	Initial seed
krylovDimension	<int>	Krylov-space dimension kdim <= kdimmax
SchurTarget	<int>	Convergence target for Krylov-Schur schur_tgt < kdim
SchurDelta	(0.1) <real>	Selection criteria for Krylov-Schur schur_del > 0
eigenTol	(1e-6) <real>	Eigenvalue tolerance eigen_tol > 0

Table 2.2. BASE_FLOW keys in the .pars file

Key	Value(s)	Description
baseFlowType	(steady) periodic mean	Base flow type
baseFlowMethod	(SFD) TDF newtown boostCOnv	Evolution operator
tolerance	(auto) <real>	

2.3 Adapting your case

git update-index –chmod +x script.sh

we recommend converting old *.rea* files to **.par** + **.re2** and **.ma2**

in *SIZE* be sure to modify and include:

```
parameter (lpelt=lelt)
include 'NEKSTAB.inc'
```

in your *.usr*, add to **userchk** and **userf **respectively:

```
call nekStab
call nekStab_forcing(ffx,ffy,ffz,ix,iy,iz,ieg)
```

2.4 Programming best practices

The best style is consistent style!

1. Always use **implicit none** with exception from *.usr*
2. Comments with ! instead of c
3. Use tab length with 3 space characters
4. small caps for fortran syntax : while, for, do, etc.
5. proper define real numbers: 0.0d0, 0.50d0, 1.0d0
6. subroutine -> return, end
7. to print output we use write(6,*)
8. stop the code with *call **exitt***
9. use *nelv* instead of **nelt** for consistency as we must respect velocity mesh
10. use *n* and *n2* for loops instead of *ntot1*
11. local field declarations with *lt* and *lt2*

```
real, save, intent(inout), dimension(lt) :: field1,field2,field3
```

1. shared variables in our custom x_SIZE.f respecting the standard Nek style like in /core

```
c     Solution data
      real vx     (lx1,ly1,lz1,lelv)
     $    ,vy     (lx1,ly1,lz1,lelv)
     $    ,vz     (lx1,ly1,lz1,lelv)
      common /vptsol/ vx, vy, vz
```

1. Limit line length increased to 132 to avoid excessive use of continuation tabs, in most cases we are slightly over the 72 characters standard

2.5 Tutorials

The following tutorials are provided to familiarize beginners with the features and typical workflow of Nek5000. For a user brand new to Nek5000, we strongly recommend beginning with the Periodic Hill tutorial.

2.5.1 Flow past a circular cylinder

This tutorial will describe how to run a case from scratch. We illustrate this procedure through a relatively simple example involving incompressible laminar flow in a two-dimensional periodic hill domain. Our implementation is loosely based on the case presented by Mellen et al. [Mellen2000]. A thorough review for this case can be found in the ERCOFTAC knowledge base wiki.

Pre-processing

We assume that you have installed Nek5000 in your home directory. This tutorial requires that you have the tools `genbox` and `genmap` compiled. Make sure `$HOME/Nek5000/bin` is in your search PATH.

Cases are setup in Nek5000 by editing case files. Users should select an editor of choice with which to do this (e.g vi). A case being simulated involves data for mesh, parameters, etc. As a first step, the user should create a case directory in their run directory.

```
cd $HOME/Nek5000/run
mkdir hillp
cd hillp
```

```
[GENERAL]
stopAt = endTime
endTime  = 200
variableDT = yes
targetCFL = 0.4
timeStepper = bdf2

[PRESSURE]
residualTol = 1e-5

[VELOCITY]
residualTol = 1e-8
```

In choosing `viscosity = -100` we are actually setting the Reynolds number. This assumes that `\rho \times u_b \times h = 1` where `u_b` denotes the bulk velocity and `h` the hill height.

We have set the calculation to stop at the physical time of `T=200` (`endTime=200`) which is roughly 22 flow-thru time units (based on the bulk velocity `u_b` and length of periodic pitch, `L=9`). Additional details on the names of keys in the `.par` file can be found here.

SIZE file

The static memory layout of Nek5000 requires the user to set some solver parameters through a so called `SIZE` file. Typically it's a good idea to start from our template. Copy the `SIZE.template` file from the core directory and rename it `SIZE` in the working directory:

```
cp $HOME/Nek5000/core/SIZE.template SIZE
```

Then, adjust the following parameters in the BASIC section

```
...

! BASIC
parameter (ldim=2)
parameter (lx1=8)
parameter (lxd=12)
parameter (lelg=22*8)
parameter (lpmax=4)
...
```

For this tutorial we have set our polynomial order to be `N=7` - this is defined in the `SIZE` file above as `lx1=8` which indices that there are 8 points in each spatial dimension of every element. Additional details on the parameters in the `SIZE` file are given here.

Compilation

With the `hillp.usr`, and `SIZE` files created, we are now ready to compile:

```
makenek hillp
```

If all works properly, upon compilation the executable `nek5000` will be generated.

Running the case

First we need to run our domain paritioning tool

```
genmap
```

On input specify `hillp` as your casename and press enter to use the default tolerance. This step will produce `hillp.ma2` which needs to be generated only once.

Now you are all set, just run

```
nekbmpi 1cyl 4
```

to launch an MPI jobs on your local machine using 4 ranks. The output will be redirected to `logfile`.

Post-processing the results

Once execution is completed your directory should now contain multiple checkpoint files that look like this:

```
hillp.f00001
hillp.f00002
...
```

The preferred mode for data visualization and analysis with Nek5000 is to use Visit/Paraview. One can use the script *visnek*, to be found in `/scripts`. It is sufficent to run:

```
visnek hillp
```

(or the name of your session) to obatain a file named `hillp.nek5000` which can be recognized in Visit/Paraview.

In the viewing window one can visualize the flow-field as depicted in `fig:hill_flow`.

2.6 References

2.6.1 2019

2.6.2 2018

2.6.3 2016

2.6.4 2015

2.6.5 2014

2.7 Source Code

2.7.1 Main code

2.7.2 Eigensolvers

2.7.3 Stabilization tools

Subroutines and functions

subroutine sfd ( )

selective frequency damping with Euler

subroutine `sfd_ab3` ( )

SFD with higher-order temporal scheme and variable time-step

Utilities

2.8 Contributing

Pull all your branches locally Create a fork of the nekStab repo: https://github.com/ricardofrantz/nekStab Add the new fork as a remote on your clone, e.g. git remote add personal https://github.com/your_user/nekStab Update branches to point at your personal repository: git branch —set-upstream-to=personal/master to track the master branch at your personal repository You can then do git push/pull as before If you want to be private then in the "Settings" on GitHub you change the visibility (it says you need to duplicate, you then need to push and pull appropriately, e.g. set "private" as the upstream repo, then push to public when you are finished with any publications for example) Submit pull requests to the central nekStab repo from your fork Of course if they have commit rights on the Central repo then they can just change their branches to point there, but it may be neater to avoid pushing directly to the central repository as we are seeking to have only the master branch publicly visible.

2.9 Changelog on Nek5000

For reference here is a log with main changes from the nek5000_V19.tar.gz

/core/prepost.f

change prepost.f line 1094 from "save nopen" to "common /RES_WANT/ nopen"

/tools/nekamg_setup/hypre/

updated the repostory path to allow compilation

- genindex
- modindex
- search

S

sfd() (fortran subroutine), 8
sfd_ab3() (fortran subroutine), 9